

Customer Support Resolution Agent

Databricks Setup & Reproduction Guide

Building the entire Databricks backend from a fresh account

Project	Customer Support Resolution Agent
Document Type	Databricks Setup / Reproduction Guide
Purpose	Recreate the full Databricks backend on a new account
Team	PwC · Cloud, Engineering & Data Analytics
Author	Akarsh
Version	1.0

1. Before You Start

This guide recreates the entire Databricks backend for the Customer Support Resolution Agent from a brand-new Databricks account. Follow it if you do not have access to the original workspace and need to build your own from scratch. By the end you will have: a catalog and schema, three Delta tables loaded with the demo data, a Genie Space, four Unity Catalog Functions, an access token, and a completed .env file the application reads.

Work through the sections in order. Each SQL block is run in the Databricks SQL Editor unless stated otherwise. After most steps there is a quick check so you can confirm it worked before moving on.

What you will end up with

- Catalog workspace, schema support_agent
- Three Delta tables: customers (15 rows), invoices (30 rows), support_tickets (24 rows)
- A Genie Space over the three tables, with business context
- Four Unity Catalog Functions: lookup_customer, apply_invoice_correction, create_case, update_account_status
- A personal access token with the sql and genie scopes
- A completed .env file

2. Create the Databricks Account

1. Go to <https://www.databricks.com/learn/free-edition> and sign up for the Free Edition. (Community Edition was retired in 2025 — use Free Edition.)
2. Complete sign-up and log in to your workspace.
3. In the left sidebar, confirm you can see Genie (under the AI/BI area) and SQL Editor. These are needed below.
4. Confirm a SQL warehouse exists: open SQL Warehouses in the sidebar. Free Edition provides a Serverless Starter Warehouse. If it is stopped, start it. This is the compute that runs all the SQL in this guide.

Note your workspace host from the browser URL — it looks like dbc-XXXXXXXX-XXXX.cloud.databricks.com. You will need it for the .env file. Do not include https:// when you record it.

3. Create the Schema and Tables

Open the SQL Editor (left sidebar). You will paste and run SQL here throughout this guide.

3.1 Create the schema

The project uses the default catalog named workspace and a schema named support_agent. Run:

```
CREATE SCHEMA IF NOT EXISTS workspace.support_agent;
```

You should see OK. If the workspace catalog does not exist in your edition, create a catalog first (Catalog Explorer → Create Catalog) and use its name consistently in place of workspace everywhere in this guide and in the .env.

3.2 Create the three tables

Paste this whole block and run it. It creates the three Delta tables with the exact schema the application expects.

```
-- Customers table
CREATE TABLE IF NOT EXISTS workspace.support_agent.customers (
  customer_id STRING,
  customer_name STRING,
  plan STRING,
```

```
    account_status STRING,  
    contact_email STRING  
);  
  
-- Invoices table  
CREATE TABLE IF NOT EXISTS workspace.support_agent.invoices (  
    invoice_id STRING,  
    customer_id STRING,  
    customer_name STRING,  
    month STRING,  
    amount_billed DOUBLE,  
    amount_expected DOUBLE,  
    status STRING,  
    issue_reason STRING  
);  
  
-- Support tickets table  
CREATE TABLE IF NOT EXISTS workspace.support_agent.support_tickets (  
    ticket_id STRING,  
    customer_id STRING,  
    customer_name STRING,  
    issue_type STRING,  
    description STRING,  
    resolution STRING,  
    status STRING,  
    created_date STRING  
);
```

Check: in Catalog Explorer, open workspace → support_agent → Tables. All three tables should be listed.

4. Load the Demo Data

Run each block below in the SQL Editor. The data is the exact synthetic dataset the project uses — 15 customers, 30 invoices, and 24 support tickets, with varied overbilling scenarios across March–May 2025.

4.1 Customers (15 rows)

```
INSERT INTO workspace.support_agent.customers VALUES
('C001', 'Acme Corp', 'Enterprise', 'Active', 'billing@acme.com'),
('C002', 'Globex Inc', 'Business', 'Active', 'accounts@globex.com'),
('C003', 'Initech Ltd', 'Starter', 'Active', 'finance@initech.com'),
('C004', 'Umbrella Co', 'Enterprise', 'Suspended', 'billing@umbrella.com'),
('C005', 'Hooli Corp', 'Business', 'Active', 'payments@hooli.com'),
('C006', 'Northwind Traders', 'Enterprise', 'Active', 'billing@northwind.com'),
('C007', 'Contoso Ltd', 'Business', 'Active', 'accounts@contoso.com'),
('C008', 'Fabrikam Inc', 'Starter', 'Active', 'finance@fabrikam.com'),
('C009', 'Tailspin Toys', 'Business', 'Suspended', 'billing@tailspin.com'),
('C010', 'Wingtip Co', 'Enterprise', 'Active', 'payments@wingtip.com'),
('C011', 'Adventure Works', 'Business', 'Active', 'billing@adworks.com'),
('C012', 'Proseware Inc', 'Starter', 'Active', 'accounts@proseware.com'),
('C013', 'Lucerne Publishing', 'Enterprise', 'Active', 'finance@lucerne.com'),
('C014', 'Coho Vineyard', 'Business', 'Active', 'billing@coho.com'),
('C015', 'Litware Inc', 'Starter', 'Suspended', 'payments@litware.com');
```

4.2 Invoices (30 rows)

```
INSERT INTO workspace.support_agent.invoices VALUES
('INV001', 'C001', 'Acme Corp', 'May 2025', 4200.00, 3500.00, 'Overbilled', 'Duplicate usage charge applied'),
('INV002', 'C001', 'Acme Corp', 'Apr 2025', 3500.00, 3500.00, 'Correct', NULL),
('INV003', 'C001', 'Acme Corp', 'Mar 2025', 3500.00, 3500.00, 'Correct', NULL),
('INV004', 'C002', 'Globex Inc', 'May 2025', 2100.00, 1800.00, 'Overbilled', 'Tax rate miscalculation'),
('INV005', 'C002', 'Globex Inc', 'Apr 2025', 1800.00, 1800.00, 'Correct', NULL),
('INV006', 'C003', 'Initech Ltd', 'May 2025', 500.00, 500.00, 'Correct', NULL),
('INV007', 'C004', 'Umbrella Co', 'May 2025', 7500.00, 6000.00, 'Overbilled', 'Legacy plan rate applied instead of negotiated rate'),
('INV008', 'C005', 'Hooli Corp', 'May 2025', 1800.00, 1800.00, 'Correct', NULL),
('INV009', 'C005', 'Hooli Corp', 'Apr 2025', 2200.00, 1800.00, 'Overbilled', 'Add-on feature billed after cancellation'),
('INV010', 'C003', 'Initech Ltd', 'Apr 2025', 500.00, 500.00, 'Correct', NULL),
('INV011', 'C006', 'Northwind Traders', 'May 2025', 9200.00, 8000.00, 'Overbilled', 'Usage tier miscalculated, charged at higher bracket'),
('INV012', 'C006', 'Northwind Traders', 'Apr 2025', 8000.00, 8000.00, 'Correct', NULL),
('INV013', 'C007', 'Contoso Ltd', 'May 2025', 2400.00, 2400.00, 'Correct', NULL),
('INV014', 'C007', 'Contoso Ltd', 'Apr 2025', 2650.00, 2400.00, 'Overbilled', 'Late fee applied in error'),
('INV015', 'C008', 'Fabrikam Inc', 'May 2025', 600.00, 600.00, 'Correct', NULL),
('INV016', 'C009', 'Tailspin Toys', 'May 2025', 3300.00, 2800.00, 'Overbilled', 'Discount not applied to renewal'),
('INV017', 'C009', 'Tailspin Toys', 'Apr 2025', 2800.00, 2800.00, 'Correct', NULL),
('INV018', 'C010', 'Wingtip Co', 'May 2025', 11000.00, 9500.00, 'Overbilled', 'Duplicate seat licenses billed'),
('INV019', 'C010', 'Wingtip Co', 'Apr 2025', 9500.00, 9500.00, 'Correct', NULL),
('INV020', 'C010', 'Wingtip Co', 'Mar 2025', 9500.00, 9500.00, 'Correct', NULL),
('INV021', 'C011', 'Adventure Works', 'May 2025', 2900.00, 2900.00, 'Correct', NULL),
```

```
( 'INV022', 'C011', 'Adventure Works', 'Apr 2025', 3200.00, 2900.00, 'Overbilled', 'Add-on
charged after downgrade'),
( 'INV023', 'C012', 'Proseware Inc', 'May 2025', 750.00, 550.00, 'Overbilled', 'Wrong plan
tier on signup'),
( 'INV024', 'C013', 'Lucerne Publishing', 'May 2025', 14500.00, 14500.00, 'Correct', NULL),
( 'INV025', 'C013', 'Lucerne Publishing', 'Apr 2025', 16000.00, 14500.00, 'Overbilled', 'Overage
charges from prior month double-counted'),
( 'INV026', 'C014', 'Coho Vineyard', 'May 2025', 2100.00, 2100.00, 'Correct', NULL),
( 'INV027', 'C014', 'Coho Vineyard', 'Mar 2025', 2500.00, 2100.00, 'Overbilled',
'Promotional rate expired early'),
( 'INV028', 'C015', 'Litware Inc', 'May 2025', 500.00, 500.00, 'Correct', NULL),
( 'INV029', 'C006', 'Northwind Traders', 'Mar 2025', 8400.00, 8000.00, 'Overbilled', 'Bandwidth
overage misreported'),
( 'INV030', 'C007', 'Contoso Ltd', 'Mar 2025', 2400.00, 2400.00, 'Correct', NULL);
```

4.3 Support tickets (24 rows)

```
INSERT INTO workspace.support_agent.support_tickets VALUES
( 'T001', 'C001', 'Acme Corp', 'Billing Dispute', 'Customer overbilled in May 2025 by 700.
Duplicate usage charge found on invoice INV001.', 'Duplicate charge identified and removed. Credit
issued.', 'Open', '2025-06-01'),
( 'T002', 'C002', 'Globex Inc', 'Billing Dispute', 'Tax rate applied incorrectly on May
invoice. Customer charged 2100 instead of 1800.', 'Tax configuration corrected. Refund of 300
initiated.', 'Resolved', '2025-06-02'),
( 'T003', 'C004', 'Umbrella Co', 'Billing Dispute', 'Enterprise customer charged legacy rate
instead of negotiated rate. Overbilled by 1500.', 'Rate table updated. Credit of 1500 applied to
account.', 'Open', '2025-06-03'),
( 'T004', 'C005', 'Hooli Corp', 'Billing Dispute', 'Add-on feature billed for April even
though customer cancelled it in March.', 'Billing rule corrected. Refund of 400 processed.',
'Resolved', '2025-05-28'),
( 'T005', 'C001', 'Acme Corp', 'Account Access', 'Customer unable to access dashboard after
billing suspension warning.', 'Access restored after billing team confirmed no overdue.',
'Resolved', '2025-05-15'),
( 'T006', 'C002', 'Globex Inc', 'Plan Upgrade', 'Customer requesting upgrade from Business
to Enterprise plan.', 'Upgrade processed. New rate effective from June 2025.', 'Resolved', '2025-
05-20'),
( 'T007', 'C003', 'Initech Ltd', 'Technical Issue', 'Customer reports slow dashboard load
times.', 'Issue escalated to infra team. Resolved in 4 hours.', 'Resolved', '2025-05-10'),
( 'T008', 'C004', 'Umbrella Co', 'Account Suspended', 'Account suspended due to repeated billing
disputes. Customer requests reinstatement.', 'Pending finance team approval.', 'Open', '2025-06-
04'),
( 'T009', 'C006', 'Northwind Traders', 'Billing Dispute', 'Customer charged at higher usage
bracket than actual consumption in May. Overbilled by 1200.', 'Usage recalculated, credit of 1200
applied.', 'Resolved', '2025-06-02'),
( 'T010', 'C007', 'Contoso Ltd', 'Billing Dispute', 'Late fee applied even though payment
was on time in April.', 'Late fee reversed after payment record check.', 'Resolved', '2025-05-
12'),
( 'T011', 'C009', 'Tailspin Toys', 'Billing Dispute', 'Renewal discount not applied,
customer overcharged by 500.', 'Discount applied retroactively, refund issued.', 'Open', '2025-06-
05'),
( 'T012', 'C010', 'Wingtip Co', 'Billing Dispute', 'Duplicate seat licenses billed in
May, overcharge of 1500.', 'Duplicate licenses removed, credit processed.', 'Open', '2025-06-06'),
( 'T013', 'C011', 'Adventure Works', 'Billing Dispute', 'Add-on feature billed after customer
downgraded plan.', 'Add-on charge reversed.', 'Resolved', '2025-05-18'),
( 'T014', 'C012', 'Proseware Inc', 'Billing Dispute', 'Customer signed up for Starter but
billed at Business tier.', 'Plan corrected, difference refunded.', 'Resolved', '2025-05-22'),
( 'T015', 'C013', 'Lucerne Publishing', 'Billing Dispute', 'Overage charges from March double-
counted in April invoice.', 'Double charge identified, 1500 credited.', 'Open', '2025-06-07'),
```

```
( 'T016', 'C014', 'Coho Vineyard',      'Billing Dispute', 'Promotional rate expired a month
earlier than agreed.', 'Promo extended, overcharge refunded.', 'Resolved', '2025-04-15'),
( 'T017', 'C008', 'Fabrikam Inc',      'Technical Issue', 'Customer reports export feature
failing intermittently.', 'Bug fixed in patch release.', 'Resolved', '2025-05-09'),
( 'T018', 'C010', 'Wingtip Co',        'Plan Upgrade',    'Customer requesting additional seats
for Q3 expansion.', 'Seats added, prorated billing applied.', 'Resolved', '2025-05-25'),
( 'T019', 'C009', 'Tailspin Toys',    'Account Suspended', 'Account suspended pending resolution
of billing dispute.', 'Awaiting customer confirmation of credit.', 'Open', '2025-06-08'),
( 'T020', 'C015', 'Litware Inc',      'Account Suspended', 'Account suspended due to non-
payment.', 'Payment plan arranged, reinstatement pending.', 'Open', '2025-06-09'),
( 'T021', 'C013', 'Lucerne Publishing', 'Plan Upgrade',    'Enterprise customer requesting custom
SLA terms.', 'Forwarded to account management team.', 'Open', '2025-06-10'),
( 'T022', 'C011', 'Adventure Works',  'Technical Issue', 'Dashboard reports not loading for
large date ranges.', 'Performance fix deployed.', 'Resolved', '2025-05-14'),
( 'T023', 'C006', 'Northwind Traders', 'Plan Upgrade',    'Customer wants to move to annual
billing for discount.', 'Annual plan activated, 10% discount applied.', 'Resolved', '2025-05-28'),
( 'T024', 'C014', 'Coho Vineyard',    'Technical Issue', 'API rate limits hit during peak
hours.', 'Rate limit increased for customer tier.', 'Resolved', '2025-05-30');
```

4.4 Verify the row counts

```
SELECT COUNT(*) FROM workspace.support_agent.customers;
SELECT COUNT(*) FROM workspace.support_agent.invoices;
SELECT COUNT(*) FROM workspace.support_agent.support_tickets;
```

You should get 15, 30, and 24. If so, the data layer is complete.

5. Create the Genie Space

Genie gives the Data Agent natural-language-to-SQL over the three tables. This is done in the Databricks UI, not in SQL.

5. In the left sidebar, click Genie (Genie Spaces). Click New (top right).
6. Title the space, e.g. Support Agent Space.
7. Under Tables/Data, select all three tables: `workspace.support_agent.customers`, `workspace.support_agent.invoices`, and `workspace.support_agent.support_tickets`.
8. For SQL Warehouse, select your Serverless Starter Warehouse.
9. Click Create. The new space opens.

5.1 Add business context

In the space's Instructions / General instructions panel, paste this so Genie interprets the data correctly:

This space contains customer support billing data.

- `customers`: customer accounts with plan type and account status.
- `invoices`: billing records. `amount_billed` is what was charged, `amount_expected` is what should have been charged. `status` is 'Correct' or 'Overbilled'. `issue_reason` explains any overbilling.
- `support_tickets`: past support cases with descriptions and resolutions.

5.2 Test the space

In the space's chat box, ask:

Which customer had the largest overbilling?

Genie should generate SQL and answer Umbrella Co, overbilled by 1500. If it does, the space works.

5.3 Copy the Space ID

Look at the browser URL while in the space:

`dbc-xxxxx.cloud.databricks.com/genie/rooms/<SPACE_ID>/chats/...`

The part right after `/rooms/` is your Genie Space ID. Copy it — it goes in the `.env`. It is an identifier, not a secret.

6. Create the Unity Catalog Functions

These four functions are what the Action Agent calls. The first is a read-only SQL function; the other three perform real writes and are Python functions. The three Python functions authenticate explicitly with a host and token, because the default credential lookup does not work inside the function sandbox.

Important: in each Python function below, replace the three placeholders before running:

- <YOUR_HOST> — your workspace host with https:// in front (e.g. https://dbc-xxxx.cloud.databricks.com)
- <YOUR_TOKEN> — the access token you create in Section 7 (create the token first if you prefer, then come back)
- <WAREHOUSE_ID> — your SQL warehouse ID (the part after /warehouses/ in the warehouse's HTTP path; see Section 7)

Security note: embedding a token in a function body is a known shortcut for this MVP. For production, replace it with a Databricks secret or service principal.

6.1 lookup_customer (read, SQL function)

```
CREATE OR REPLACE FUNCTION workspace.support_agent.lookup_customer(customer_name STRING)
RETURNS TABLE(customer_id STRING, customer_name STRING, plan STRING,
               account_status STRING, contact_email STRING)
COMMENT 'Look up a customer account by name.'
RETURN
  SELECT customer_id, customer_name, plan, account_status, contact_email
  FROM workspace.support_agent.customers
  WHERE LOWER(customer_name) LIKE LOWER(CONCAT('%', lookup_customer.customer_name, '%'));
```

Test:

```
SELECT * FROM workspace.support_agent.lookup_customer('Acme');
```

6.2 apply_invoice_correction (write, Python function)

```
CREATE OR REPLACE FUNCTION workspace.support_agent.apply_invoice_correction(
  p_invoice_id STRING,
  p_corrected_amount DOUBLE
)
RETURNS STRING
LANGUAGE PYTHON
COMMENT 'Apply a billing correction to an invoice.'
AS $$
  from databricks.sdk import WorkspaceClient
  w = WorkspaceClient(host="<YOUR_HOST>", token="<YOUR_TOKEN>")
  w.statement_execution.execute_statement(
    warehouse_id="<WAREHOUSE_ID>",
    statement=f"""
      UPDATE workspace.support_agent.invoices
      SET amount_billed = {p_corrected_amount}, status = 'Corrected'
      WHERE invoice_id = '{p_invoice_id}'
    """
  )
  return f"Invoice {p_invoice_id} corrected to {p_corrected_amount} and marked Corrected."
$$;
```

Test (then check the row changed):

```
SELECT workspace.support_agent.apply_invoice_correction('INV011', 8000.0);
SELECT invoice_id, amount_billed, status FROM workspace.support_agent.invoices WHERE invoice_id = 'INV011';
```


6.3 create_case (write, Python function)

```
CREATE OR REPLACE FUNCTION workspace.support_agent.create_case(
  p_customer_id STRING,
  p_customer_name STRING,
  p_issue_type STRING,
  p_description STRING
)
RETURNS STRING
LANGUAGE PYTHON
COMMENT 'Create a real support case by inserting a new row into support_tickets.'
AS $$
    from databricks.sdk import WorkspaceClient
    w = WorkspaceClient(host="<YOUR_HOST>", token="<YOUR_TOKEN>")
    new_id = "T" + str(abs(hash(p_customer_id + p_description)) % 100000)
    desc = p_description.replace("'", "")
    w.statement_execution.execute_statement(
        warehouse_id="<WAREHOUSE_ID>",
        statement=f"""
            INSERT INTO workspace.support_agent.support_tickets
            (ticket_id, customer_id, customer_name, issue_type, description, resolution, status,
created_date)
            VALUES ('{new_id}', '{p_customer_id}', '{p_customer_name}', '{p_issue_type}',
'{desc}', NULL, 'Open', current_date())
        """
    )
    return f"Case {new_id} created for {p_customer_name} - type: {p_issue_type}, status: Open."
$;
```

Test (then check the row exists):

```
SELECT workspace.support_agent.create_case('C001', 'Acme Corp', 'Billing Dispute', 'Overbilled in
May 2025');
SELECT * FROM workspace.support_agent.support_tickets WHERE customer_id = 'C001' ORDER BY
created_date DESC;
```

6.4 update_account_status (write, Python function)

```
CREATE OR REPLACE FUNCTION workspace.support_agent.update_account_status(
  p_customer_id STRING,
  p_new_status STRING
)
RETURNS STRING
LANGUAGE PYTHON
COMMENT 'Update a customer account status.'
AS $$
    from databricks.sdk import WorkspaceClient
    w = WorkspaceClient(host="<YOUR_HOST>", token="<YOUR_TOKEN>")
    w.statement_execution.execute_statement(
        warehouse_id="<WAREHOUSE_ID>",
        statement=f"""
            UPDATE workspace.support_agent.customers
            SET account_status = '{p_new_status}'
            WHERE customer_id = '{p_customer_id}'
        """
    )
    return f"Account {p_customer_id} status updated to {p_new_status}."
```

```
$$;
```

Test (then check it changed):

```
SELECT workspace.support_agent.update_account_status('C009', 'Active');
```

```
SELECT customer_id, account_status FROM workspace.support_agent.customers WHERE customer_id =  
'C009';
```

7. Access Token, Warehouse ID, and .env

7.1 Find the warehouse HTTP path and ID

10. Open SQL Warehouses → your Serverless Starter Warehouse → Connection details.
11. Copy the HTTP Path. It looks like /sql/1.0/warehouses/abc123def456.
12. The part after /warehouses/ (e.g. abc123def456) is your warehouse ID — used in the Python UC Functions and to build the .env.

7.2 Create the access token (with sql and genie scopes)

13. Top-right profile icon → Settings → Developer → Access tokens → Manage.
14. Generate new token. Give it a name (e.g. support-agent) and an expiry.
15. Ensure the token includes BOTH the sql and genie scopes. The fixed SQL tools use sql; Genie uses genie. If you only grant one, the other capability fails.
16. Copy the token now (it is shown only once). It begins with dapi...

If you created the UC Functions in Section 6 with a placeholder token, go back now and replace <YOUR_TOKEN> in each Python function with this token, then re-run those CREATE statements.

7.3 Build the .env file

Create a file named .env in the project root with these keys, filled in with your values:

```
DATABRICKS_HOST=dbc-xxxxxxx-xxxx.cloud.databricks.com
DATABRICKS_TOKEN=dapi...your-token
DATABRICKS_HTTP_PATH=/sql/1.0/warehouses/your-warehouse-id
DATABRICKS_CATALOG=workspace
DATABRICKS_SCHEMA=support_agent
DATABRICKS_GENIE_SPACE_ID=your-genie-space-id
GOOGLE_API_KEY=your-gemini-api-key
```

Notes: DATABRICKS_HOST has no https:// (bare hostname). No quotes around values. The .env is gitignored and must never be committed. The GOOGLE_API_KEY is a Google Gemini key from <https://aistudio.google.com> — if a new key gives a 401, use an Alza-prefixed key or update the google client library.

8. Final Verification

With the backend built, confirm everything from the application side:

17. From the backend folder (virtual environment active), run `python test_connection.py` — it should connect and list customers.
18. Run `python test_sql_mcp.py` with the SQL MCP server running — the fixed tools and Genie should return results.
19. Run `python test_action_mcp.py` with the Action MCP server running — all four UC Function tools should return results and the writes should be visible in the tables.

Once those pass, the new Databricks backend is a faithful reproduction of the original, and the application will run against it using the .env you built. From here, follow the project README to install and run the four processes.